Week 04 — Practice 1: Processes and Threads

> Practice questions with the instructor's model answers (marked **R/** on the slides). Questions: practice-1.pdf — filed under `exercises/` because it is in-class practice, not a lecture. Theory behind it: week 03 notes (Chapter II: Process Management).

Six scenarios, each one testing whether you can tell a **process-level** fact from a **thread-level** one. That distinction is the whole point of this practice.

Question 1 — The frozen word processor

> A user opens a very large document and the **whole application** becomes unresponsive: the window won't repaint, menus won't open, clicking does nothing. But the **OS is still fully responsive** (other apps work, the mouse moves, the clock ticks).

a) Is this a process-level or thread-level failure? Justify.

R/ This is a **thread-level failure**, not a process-level one. The word processor **process is still alive** (it occupies memory, the OS still tracks it), but the **thread handling the UI events is blocked**, likely waiting on a **synchronous I/O** operation (loading the large document) or stuck in a **long computation**. Since the UI thread is blocked, the window cannot repaint or respond to input.

b) What does the OS's continued responsiveness tell you about the relationship between the word processor and the OS?

R/ The OS is a **separate set of processes** (window manager, scheduler, input handlers) that are **independent** of the word processor. The word processor blocking does not block the OS because they run in **different address spaces** and are **scheduled independently**.

c) If the application had been designed with multiple threads, how would the behavior differ?

R/ With **one thread for the UI and another for loading/processing** the document, the UI thread would stay free to repaint the window and respond to input while the background thread works. The application would **appear responsive even during heavy work**.

> **Link to the week 03 lab:** this is exactly why the CPU dashboard uses `root.after(1000, update)` rather than a sleeping loop — the UI thread must never be the one that waits.

Question 2 — Entries with status Z

> The process table fills with entries showing status **Z**. They consume no CPU or memory but accumulate, and eventually the system cannot create new processes.

a) Is it a process or a thread? Be specific about the case and how it happens.

R/ It is a **zombie process**: a process that has **terminated** but whose **parent has not yet called** `wait()` **or** `waitpid()` to collect its exit status. The process is dead and holds no memory or CPU, but **its entry remains in the process table**.

b) Why does it still occupy an entry in the process table if it's "dead"?

R/ The entry is retained because it holds the **child's exit status** and other termination information that the **parent must read**. Until the parent reads it, the OS cannot free the entry.

c) What is the parent doing wrong, and how would you fix it?

R/ The parent is **not calling** `wait()` on its children. Fix: call `wait()` **or** `waitpid()` **after forking**, or install a `SIGCHLD` **handler** that reaps children automatically.

d) Could this happen with threads? Why or why not?

R/ No. Threads do not become zombies in the same sense. When a thread finishes, its resources are **reclaimed by the process**, because it shares the process's address space and **exit status is handled at the process level**, not per thread. Threads can be **joinable or detached**, but that is a different mechanism — there is **no kernel-level zombie thread state**.

> Why it's dangerous even though zombies use no memory: the **process table is a finite resource**. Running out of entries means **no new processes can be created at all**.

Question 3 — Chrome with 15 tabs

> Task Manager shows **15+ separate** `chrome.exe` **processes**, each with its own memory. Closing one tab kills exactly one process without affecting the others.

a) Why would a browser use multiple processes instead of multiple threads for tabs?

R/ Chrome uses multiple processes (**one per tab**, plus a browser process and a GPU process) for **isolation and security**. Each tab runs in its **own address space**, so a **crash or exploit in one tab cannot affect others**.

b) What is the primary advantage of this design?

R/ Fault isolation and security. A malicious or buggy page **cannot read another tab's memory**, and a crash in one tab doesn't bring down the whole browser.

c) What is the primary disadvantage compared to a multithreaded design?

R/ Resource overhead. Each process has its own **address space, page tables, file descriptors and kernel structures**, consuming more memory and making **context switches more expensive** than threads.

d) If one tab crashes, why don't the others crash? Explain in terms of address spaces.

R/ Each tab runs in its **own address space**. When one process crashes, the OS **tears down only that process's memory**; the others are unaffected because they have **separate page tables** and cannot access the crashed process's memory.

Question 4 — One process per request vs one thread per request

> **Design A:** one process per request. **Design B:** one thread per request (single process). B handles **significantly more requests per second**, but A is **more resilient** — a crash in one request doesn't take down the others.

a) Why are thread creation and context switching cheaper than for processes?

R/ Because threads **share the same address space**, the OS does **not** need to:

- create a new page table or **switch page tables**,
- duplicate or copy **file descriptor tables**,
- set up a **new address space**.

A **process switch** requires switching page tables (and often **flushing the TLB**), which is expensive. A **thread switch** inside the same process only swaps **registers, stack pointer, and program counter**.

b) Why does Design A survive a crash that would kill Design B?

R/ In A each request runs in its **own address space**, so a crash (segfault, memory corruption, unhandled exception) makes the OS **kill only that process** while the others continue. In B **all threads share one address space**, so a crash in one thread (e.g. dereferencing a bad pointer) typically **brings down the entire process — all threads die**.

d) If you needed both performance and isolation, what architectural approach would you choose?

R/ A **hybrid**: a **process pool** where each process handles multiple requests using a **thread pool**. This gives **isolation between processes** while keeping **thread-level efficiency** inside each one. This is essentially the model used by modern web servers (Apache's prefork + worker MPM, nginx, etc.).

(The slides skip a part (c) in this question and go from (b) to (d).)

Question 5 — The lost increments

> Two threads each increment a shared global `counter` one million times. The expected final value is **2,000,000**, but the result is lower (e.g. **1,347,219**) and **changes every run**.

a) Explain precisely why the result is less than expected.

R/ `counter++` is **not atomic**. It compiles into **three steps**: **load** counter into a register, **increment** the register, **store** it back. If two threads interleave, **both load the same value, both increment, both store — one increment is lost**. Over a million iterations, thousands of increments are lost.

b) What is the name of this phenomenon?

R/ A **race condition**.

c) Is this a problem with threads, with the OS scheduler, or with the programmer? Justify.

R/ A **programmer problem**. Threads and the scheduler behave correctly; the programmer **failed to synchronize access to shared data**. The OS provides primitives (**mutexes, semaphores, atomics**) but **does not automatically protect shared variables**.

d) What if the same code ran as two separate processes with shared memory?

R/ The problem would persist. Shared memory across processes is subject to the **same race conditions**. It would not occur if each process had its **own private copy** — but then they wouldn't be incrementing the same variable, and the result would be **1,000,000 per process**, not 2,000,000 total.

> The result being **different every run** is the signature of a race condition: the outcome depends on scheduling, which you don't control.

Question 6 — Globals are visible, locals are not

> Thread A modifies a **global** variable and Thread B immediately sees the change. But Thread B **cannot see** a **local** variable declared inside a function of Thread A, even though both threads are in the same process.

a) Why can Thread B see the global but not the local?

R/ Globals live in the process's data segment, which is **shared by all threads**. **Locals live on each thread's own stack**, which is **private to that thread**. B can see the global because it's in shared memory; it cannot see A's local because that local is on **A's stack**.

b) What parts of memory are shared between threads, and what parts are private?

Shared between threads	Private to each thread
Code segment	Stack
Data segment (globals)	Registers
Heap	Program counter
Open file descriptors	Thread-local storage (TLS)
Current working directory	
Signal handlers	

c) If these were two separate processes, what would change about sharing the global?

R/ The global would **not be shared by default**. Each process would have its **own copy** in its own address space. To share, the programmer needs **explicit IPC** (shared memory, pipes, message queues, etc.).

d) Why is this shared-memory model both a benefit and a danger?

R/ Benefit: communication is **fast** — no kernel involvement, no copying, no serialization. **Danger: any thread can corrupt shared data**, leading to race conditions, inconsistent state and bugs that are hard to reproduce. It **shifts the whole burden of synchronization onto the programmer**.

Most important for exams

- **Diagnosis rule:** if the **app** freezes but the **OS** is fine → a **blocked thread**, not a dead process. Different address spaces + independent scheduling is why the OS keeps running.
- **Fix for a frozen UI:** separate the **UI thread** from the **work thread**.

- **Zombie:** terminated child, exit status not yet collected; still holds a **process table entry**. Caused by a parent that never calls `wait()` / `waitpid()`; fixed by calling it or handling `SIGCHLD`. **Threads have no zombie state** (joinable/detached is a different mechanism). The real damage is **exhausting the process table**.
- **Chrome = process per tab:** advantage **fault isolation and security**; disadvantage **memory and context-switch overhead**. Separate **address spaces and page tables** are why one crash stays contained.
- **Why thread switches are cheap:** no new page table, no page-table switch (**no TLB flush**), no copied file descriptor table — only **registers, stack pointer, PC**.
- **Process-per-request = resilient; thread-per-request = fast; the answer in practice is a hybrid process pool + thread pool.**
- **Race condition:** `counter++` is **load, increment, store** — not atomic. It's the **programmer's** fault; the OS only offers **mutexes, semaphores, atomics**. **Shared memory between processes races too.**
- **Shared vs private memory for threads:** shared = **code, data (globals), heap, file descriptors, cwd, signal handlers**; private = **stack, registers, PC, TLS**. This table answers many exam questions by itself.
- **Separate processes share nothing by default — sharing needs explicit IPC.**